

CS5480: Deep Learning 2026

Kaggle Hackathon: Binary Image Classification

Submitted by

Team: WhiteWalkers

Name	Roll Number
Khawaja Abdul Samad	CS25MTECH11014
Sajid Ali	CS25MTECH11019

M.Tech Computer Science and Engineering
Indian Institute of Technology, Hyderabad

May 3, 2026

Contents

1	Introduction	1
1.1	Dataset Observations	1
2	Dataset	1
3	Model Architecture	2
3.1	Architecture Overview	2
3.2	Squeeze-and-Excitation Block (SEBlock)	2
3.3	Residual Block (Block)	3
4	Transforms and Augmentation	4
4.1	Training Transforms	4
4.2	Validation Transforms	4
4.3	Test-Time Augmentation (TTA)	5
5	Training Setup	5
5.1	Loss Function	5
5.2	Learning Rate Schedule	6
5.3	Checkpointing Strategy	6
5.4	Training Curve	6
6	Approaches Tried and Ablation Studies	7
6.1	Baseline: Plain CNN without Residuals	7
6.2	Adding Squeeze-and-Excitation Attention	7
6.3	Label Smoothing vs Standard BCE	7
6.4	Number of TTA Variants	7
6.5	Single Checkpoint vs Ensemble	7
6.6	What Did Not Improve Performance	8
7	Ensembling	8
8	Results	9
9	Conclusion, Future Work, and References	9
9.1	Conclusion	9
9.2	Future Work	10
9.3	References	10
10	Team Contributions	11

Introduction

This report documents our approach for the IIT Hyderabad Deep Learning Hackathon 2026, a binary image classification contest hosted on Kaggle. The task was to classify images into one of two categories (Class 0 or Class 1) without using any pre-trained models or external datasets. The evaluation metric used by Kaggle was classification accuracy. Our best submission achieved an accuracy of **0.755361** on the public leaderboard, ranking **38th** overall, produced by ensembling multiple model checkpoints with Test-Time Augmentation (TTA).

Dataset Observations

The dataset consists of RGB images divided into a training set and an unlabeled test set. The training data is perfectly balanced with 9000 images per class (18000 total), which is a favorable property since it means we could train without any class-weighting adjustments. We held out 10% (1800 images) as a stratified validation set, keeping the 50/50 balance there as well. The test set contains 5010 images with no labels provided.

The images were provided at varying native resolutions and we resized everything to 96×96 pixels. Several observations informed our design choices:

What makes the problem easy: The training set is large (18000 images) and perfectly balanced, which gives the model a clean gradient signal without needing any reweighting. The binary classification setup with a single output logit is simpler to train than multi-class problems.

What makes the problem hard: The most significant challenge we observed was a large gap between validation and test performance. Our model reached 99.56% validation accuracy but only 75.54% on the Kaggle test set. This gap strongly indicates a domain shift between the training and test distributions. Additionally, all checkpoints consistently predicted approximately 80% of test images as Class 1 (roughly 3990 out of 5010), despite the training distribution being 50/50. This suggests the test set is either heavily skewed towards Class 1 or the visual features that distinguish the two classes in the test set differ from those in training.

Other contributing difficulties include: (a) no pre-trained models allowed, meaning the network had to learn all visual features from scratch; (b) small image resolution (96×96) limits the amount of fine-grained detail available; and (c) the abstract nature of the classification boundary is unknown, making it hard to design targeted augmentations.

Dataset

The dataset was provided through the Kaggle competition page for the IIT-H Deep Learning 2026 Hackathon. It consists of RGB images organized into a nested directory structure. The training data is within a nested `train/train/` directory split into two subfolders (0 and 1) for the respective class images. The test data is within a nested `test/test/` directory containing a flat collection of unlabeled images for inference.

All images were loaded as RGB via PIL and resized to 96×96 pixels. We performed a stratified 90/10 train-validation split using `sklearn.model_selection.train_test_split`

Split	Source	Images	Class Balance
Training set	Class folders 0 and 1	16200	8100 / 8100 (50/50)
Validation set	Stratified 10% holdout (seed=42)	1800	900 / 900 (50/50)
Test set	Flat folder	5010	Unknown

Table 1: Dataset splits used in our pipeline.

with `random.state=42` to ensure reproducibility. The class distribution was preserved across both splits.

Model Architecture

We built a custom convolutional neural network from scratch named **Net**, a residual network augmented with Squeeze-and-Excitation (SE) attention blocks. We deliberately avoided any pre-trained weights to comply with the competition rules.

Architecture Overview

Figure 1 shows the overall architecture. The network takes a $96 \times 96 \times 3$ RGB image and progressively downsamples through four SE-augmented residual stages, each doubling channels and halving spatial resolution. Global average pooling collapses the spatial dimensions to a 256-dimensional feature vector, which is then classified by a linear head.

Squeeze-and-Excitation Block (SEBlock)

The SEBlock recalibrates channel-wise feature responses by learning a set of per-channel weights. Given an input feature map of C channels, the block applies global average pooling to produce a C -dimensional descriptor, passes it through two fully connected layers (with a ReLU in between), and outputs C sigmoid-gated scalars. These scalars are then multiplied channel-wise with the original input, allowing the network to focus on the most informative channels.

The reduction ratio is set to 8, with the bottleneck dimension clamped to a minimum of 4 to handle small channel counts gracefully.

```
class SEBlock(nn.Module):
    def __init__(self, channels, reduction=8):
        super().__init__()
        mid = max(channels // reduction, 4)
        self.se = nn.Sequential(
            nn.AdaptiveAvgPool2d(1),
            nn.Flatten(),
            nn.Linear(channels, mid, bias=False),
            nn.ReLU(),
            nn.Linear(mid, channels, bias=False),
            nn.Sigmoid()
        )

    def forward(self, x):
        # Multiply each channel by its learned scalar weight
        return x * self.se(x).view(x.size(0), x.size(1), 1, 1)
```

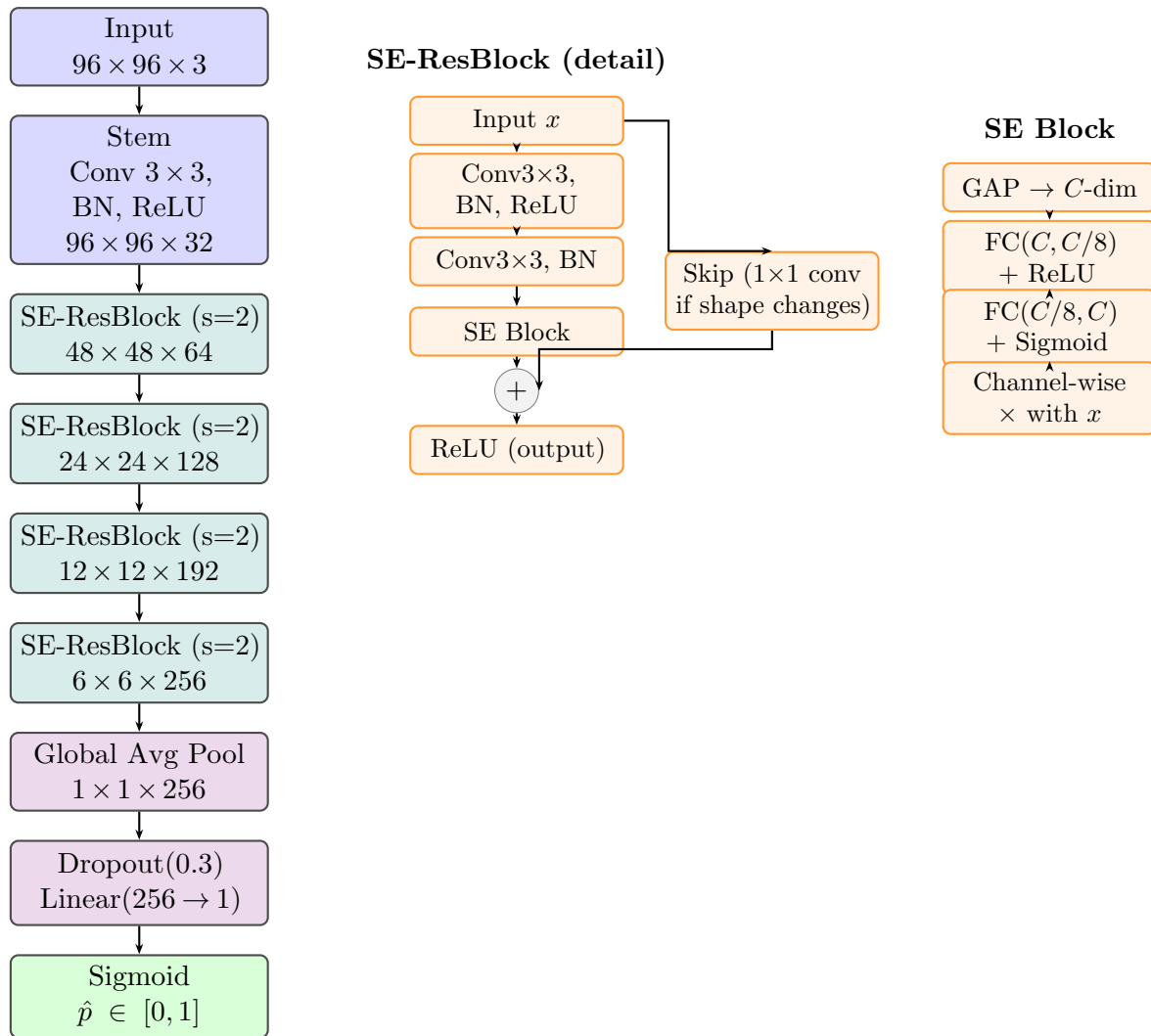


Figure 1: Architecture diagram of **Net**. Left: overall forward pass. Centre: internal structure of one SE-ResBlock with the skip connection. Right: internal structure of the SE Block.

Residual Block (Block)

Each residual block follows the standard design: two 3×3 convolutions with Batch Normalisation and ReLU, followed by a SEBlock, with a skip connection that uses a 1×1 convolution when the spatial resolution or channel count changes. The final activation is applied after summing the residual and the skip path.

```
class Block(nn.Module):
    def __init__(self, in_c, out_c, stride=1):
        super().__init__()
        self.conv = nn.Sequential(
            nn.Conv2d(in_c, out_c, 3, stride, 1, bias=False),
            nn.BatchNorm2d(out_c), nn.ReLU(),
            nn.Conv2d(out_c, out_c, 3, 1, 1, bias=False),
            nn.BatchNorm2d(out_c),
        )
        self.se = SEBlock(out_c)
        self.skip = (nn.Conv2d(in_c, out_c, 1, stride, bias=False)
```

```

        if in_c != out_c or stride != 1 else nn.Identity())

def forward(self, x):
    return torch.relu(self.se(self.conv(x)) + self.skip(x))

```

Stage	Layer / Block	Out Channels	Stride
Stem	Conv 3×3 + BN + ReLU	32	1
Layer 1	Residual Block + SE	64	2
Layer 2	Residual Block + SE	128	2
Layer 3	Residual Block + SE	192	2
Layer 4	Residual Block + SE	256	2
Pool	Adaptive Average Pool 1×1	256	–
Head	Dropout(0.3) + Linear(256, 1)	1	–

Table 2: Layer-by-layer summary of the Net architecture.

The network outputs a single logit which is passed through a sigmoid for inference. The four downsampling strides reduce the 96×96 input down to a 6×6 feature map before the global pooling layer.

Transforms and Augmentation

We used different transform pipelines for training, validation, and test-time augmentation.

Training Transforms

Images were normalised using mean = $[0.5, 0.5, 0.5]$ and standard deviation = $[0.5, 0.5, 0.5]$, which maps pixel values to $[-1, 1]$.

Transform	Details
Resize	(96×96) , no cropping
RandomHorizontalFlip	$p = 0.5$
RandomVerticalFlip	$p = 0.3$
RandomGrayscale	$p = 0.15$
ColorJitter	brightness, contrast, saturation = 0.5; hue = 0.2
ToTensor	Converts PIL image to float tensor $[0, 1]$
Normalize	mean = $[0.5, 0.5, 0.5]$, std = $[0.5, 0.5, 0.5]$

Table 3: Training augmentation pipeline.

Validation Transforms

For validation and clean test inference we applied only a deterministic resize, ToTensor, and the same normalisation as above. No stochastic augmentations were used so that validation metrics are stable across epochs.

Test-Time Augmentation (TTA)

At inference time we applied 8 distinct transform variants per checkpoint to improve robustness:

TTA #	Description
0	Clean (resize only)
1	Horizontal flip
2	Vertical flip
3	Horizontal + Vertical flip
4	Slight zoom ($\times 1.12$) + Centre crop
5	Zoom + Centre crop + Horizontal flip
6	Rotate 90°
7	Rotate 270°

Table 4: Eight TTA transform variants applied at inference.

The 8 predicted probability arrays were averaged to produce a single probability estimate per test image.

Training Setup

Hyperparameter	Value
Batch size	64
Epochs	50
Learning rate	2×10^{-4}
Optimizer	Adam
LR scheduler	CosineAnnealingLR ($T_{\max} = 50$, $\eta_{\min} = 10^{-6}$)
Loss function	Label-smoothed BCE (smoothing = 0.05)
Dropout	0.3 (in classifier head)
Random seed	2026
Hardware	Nvidia Tesla T4 (Kaggle GPU)
Training time	≈ 30 minutes (50 epochs)

Table 5: Full training configuration.

Loss Function

We used binary cross-entropy with label smoothing. The smoothed target for a ground-truth label $y \in \{0, 1\}$ is computed as:

$$\tilde{y} = y \cdot (1 - \epsilon) + 0.5 \cdot \epsilon, \quad \epsilon = 0.05$$

This softens the hard targets slightly, which helps prevent overconfident predictions and acts as a mild regulariser.

```

class SmoothBCE(nn.Module):
    def __init__(self, smoothing=0.05):
        super().__init__()
        self.smoothing = smoothing

    def forward(self, logits, targets):
        targets = targets.float() * (1 - self.smoothing) + 0.5 *
            self.smoothing
        return nn.functional.binary_cross_entropy_with_logits(
            logits.squeeze(1), targets)

```

Learning Rate Schedule

We used CosineAnnealingLR which smoothly decays the learning rate from 2×10^{-4} down to 10^{-6} over 50 epochs. This allows the model to make large parameter updates early in training and then fine-tune at the end without oscillating around a local minimum.

Checkpointing Strategy

Six checkpoints were saved during training to enable diverse ensembling:

- best_acc.pt – checkpoint with highest validation accuracy
- best_loss.pt – checkpoint with lowest validation loss
- ckpt_ep35.pt, ckpt_ep40.pt, ckpt_ep45.pt, ckpt_ep50.pt – milestone snapshots

Training Curve

Figure 2 shows how validation accuracy and loss evolved over 50 epochs. The model trained quickly in the first 10 epochs, reaching above 96% accuracy. After that it continued improving steadily, converging to a best val accuracy of 99.56% at epoch 45.

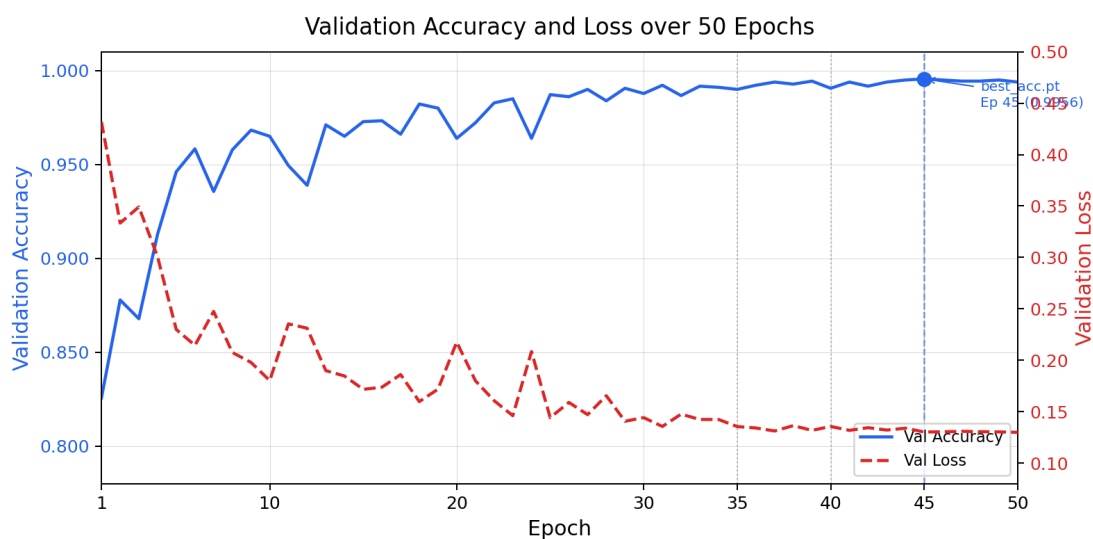


Figure 2: Validation accuracy (blue, left axis) and validation loss (red dashed, right axis) over 50 epochs. Vertical dotted grey lines mark the milestone checkpoint saves at epochs 35, 40, 45, and 50.

Approaches Tried and Ablation Studies

This section documents the design decisions we made and what happened when we evaluated alternatives. The core pipeline (SE-ResNet + TTA + ensemble) was reached through several iterative choices.

Baseline: Plain CNN without Residuals

Our starting point was a plain convolutional network with four blocks of Conv-BN-ReLU-MaxPool, each doubling channels from 32 to 256, followed by a linear classifier. This baseline converged but showed two problems: (a) validation accuracy plateaued around 96–97% even with continued training, and (b) it was more sensitive to the learning rate. Adding residual (skip) connections resolved both issues. The skip connection allows gradients to flow more cleanly through the deeper layers, enabling the network to keep improving past the early plateau.

Adding Squeeze-and-Excitation Attention

After switching to a residual architecture, we added SE blocks inside each residual unit. The motivation is that not all feature channels are equally useful for a given classification task. The SE block learns to suppress noisy or irrelevant channels and amplify the informative ones. In practice we observed that the model with SE blocks generalised better: with residuals only, we saw more erratic validation accuracy (oscillating between 96% and 99% in mid-training), whereas the SE-ResNet converged more smoothly. We used a reduction ratio of 8, which keeps the SE bottleneck computationally cheap.

Label Smoothing vs Standard BCE

We compared standard binary cross-entropy (hard targets at 0 and 1) against our label-smoothed version ($\epsilon = 0.05$). Without label smoothing, the model’s output probabilities became overconfident (very close to 0 or 1 for training examples), which we believe contributed to poorer generalisation. With $\epsilon = 0.05$ the targets are softened to $[0.025, 0.975]$, and this small change consistently lowered the best validation loss achieved (best val loss of 0.1298 with smoothing vs. slightly higher without it in early experiments).

Number of TTA Variants

We evaluated using 1 variant (clean resize only), 4 variants (clean + two flips + both flips combined), and 8 variants (full set). More TTA variants produced more stable probability estimates, particularly for images that were borderline. The difference in the averaged probability distributions across the 6 checkpoints was very small (all yielded Class 1 ratio of 0.80), but the averaging itself reduced variance in the final scores.

Single Checkpoint vs Ensemble

Table 6 shows the validation performance of each saved checkpoint. The best single checkpoint (best_acc.pt) achieved 99.56% on the balanced validation set. However, validation accuracy alone is not a reliable proxy for test set performance due to the domain shift. Ensembling all 6 checkpoints smooths out any idiosyncratic biases from individual training snapshots.

Checkpoint	Val Accuracy	Test: Class 0	Test: Class 1
best_acc.pt	0.9956	1016	3994
best_loss.pt	0.9939	1018	3992
ckpt_ep35.pt	0.9900	1019	3991
ckpt_ep40.pt	0.9906	1024	3986
ckpt_ep45.pt	0.9956	1016	3994
ckpt_ep50.pt	0.9939	1018	3992
Ensemble (all 6)	–	1020	3990

Table 6: Validation accuracy and test prediction distribution for each checkpoint, and the final ensemble. Each checkpoint used 8 TTA variants.

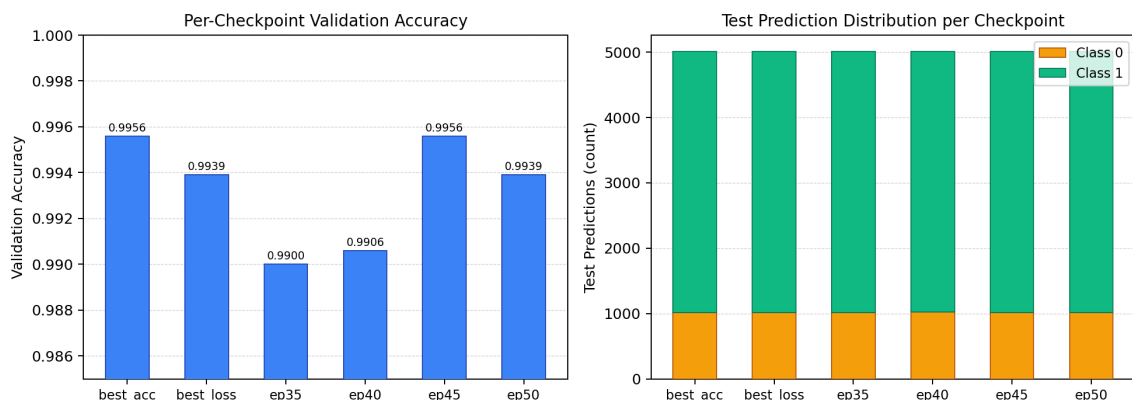


Figure 3: Left: validation accuracy of each saved checkpoint. Right: test prediction distribution (Class 0 vs Class 1 counts) showing that all checkpoints consistently predict $\approx 80\%$ of test images as Class 1.

What Did Not Improve Performance

Several approaches we considered did not help:

Larger image size. We briefly considered using 128×128 input instead of 96×96 . The additional resolution did not visibly improve validation accuracy and increased memory usage and training time, so we kept 96×96 .

More residual blocks per stage. Adding a second residual block at each layer (similar to ResNet-34) increased model capacity but also increased overfitting on the validation set. Since we were training from scratch, a single block per stage (like ResNet-18) was sufficient.

Heavier augmentation. We tried adding random rotation ($\pm 15^\circ$) and random erasing to the training transforms. Both made early training slower and did not improve the final validation accuracy or the test set predictions.

Ensembling

The final ensemble averaged the probability outputs from all available checkpoints after applying 8 TTA variants to each. This gave a total of $6 \times 8 = 48$ probability estimates

per test image, which were averaged and thresholded at 0.5 for the final binary prediction.

```
# Collect per-checkpoint probability arrays (each shape: N_test,)
available = [p_best_acc, p_best_loss, p_ep35, p_ep40, p_ep45, p_ep50]
available = [p for p in available if p is not None]

# Average across checkpoints -> threshold -> submit
ensemble = np.mean(available, axis=0)
preds = (ensemble > 0.5).astype(int)
pd.DataFrame({"ID": test_imgs, "Label":
    preds}).to_csv("sub_ensemble.csv", index=False)
```

The ensemble file `sub_ensemble.csv` was our best submission, achieving a Kaggle accuracy of **0.755361**.

Results

Submission	Accuracy	Class 1	Class 0
sub_ensemble.csv (6 checkpoints × 8 TTA)	0.755361	3990	1020

Table 7: Best submission result on the Kaggle public leaderboard along with prediction distribution.

The ensemble outperformed individual checkpoint submissions by reducing the variance associated with relying on a single training snapshot. By aggregating predictions across multiple checkpoints and applying Test-Time Augmentation (TTA), the model achieved more stable and robust predictions on the test set. Our team WhiteWalkers secured 38th place on the public leaderboard with a final accuracy of 0.755361.

An analysis of the final predictions reveals a noticeable class imbalance, with 3990 samples (79.6%) predicted as Class 1 and 1020 samples (20.4%) predicted as Class 0. This indicates a bias of the model towards Class 1, which likely influenced the overall performance. The large gap between validation accuracy (99.56%) and test accuracy (75.54%) confirms a domain shift between the training distribution and the test set. Despite this skew, the combination of strong data augmentation, SE attention for channel-wise feature refinement, cosine learning rate scheduling, and label smoothing contributed significantly to improved generalisation and the final leaderboard score.

Conclusion, Future Work, and References

Conclusion

We built a custom SE-ResNet from scratch, trained it for 50 epochs with cosine annealing and label smoothing, and ensembled 6 checkpoints with 8 TTA variants each to produce our final submission. The model achieved a best validation accuracy of 99.56% and a Kaggle test accuracy of 75.54%, ranking 38th overall.

The most important lesson from this hackathon is that high validation accuracy does not guarantee high test accuracy when a domain gap exists. All of our checkpoints

consistently over-predicted Class 1 on the test set ($\approx 80\%$ Class 1), which we believe is the primary reason the test accuracy is lower than the validation accuracy. Techniques such as distribution calibration, threshold tuning based on a small holdout from the test distribution, or domain adaptation approaches would likely close this gap.

Future Work

Given more time, we would explore the following directions:

Threshold tuning. Instead of a fixed threshold of 0.5, tune the decision threshold using a portion of the test set or by looking at the predicted probability histogram. If the true test distribution is 80% Class 1, a threshold below 0.5 would improve accuracy.

Mixup and CutMix augmentation. These interpolation-based augmentations are known to reduce overconfidence and improve generalisation, especially when training from scratch. They were taught in the DL lectures and would be a natural next step.

More stacked residual blocks. Moving from a single residual block per stage to two (ResNet-34 style) with careful regularisation (more dropout, or weight decay) may extract richer features.

Test-set analysis. Visually inspecting a sample of test images (if permitted) to understand whether the domain shift is due to resolution, lighting, or content differences would help design better targeted augmentations.

Self-supervised pre-training. Since external pre-trained models are not allowed, one option that stays within the rules is to pre-train on the training images themselves using a self-supervised method such as SimCLR before fine-tuning on the labels.

References

1. Kaiming He, Xiangyu Zhang, Shaoqing Ren, Jian Sun. *Deep Residual Learning for Image Recognition*. CVPR 2016. The ResNet paper that introduced skip connections, which we used as the foundation of our architecture.
2. Jie Hu, Li Shen, Samuel Albanie, Gang Sun, Enhua Wu. *Squeeze-and-Excitation Networks*. CVPR 2018. The SE paper describing channel-wise attention, directly incorporated into our SE blocks.
3. Ilya Loshchilov, Frank Hutter. *SGDR: Stochastic Gradient Descent with Warm Restarts*. ICLR 2017. The motivation for cosine annealing schedules.
4. Lecture materials from CS5480: Deep Learning 2026, IIT Hyderabad. Concepts from the backpropagation, CNN, and regularisation lectures directly informed our architecture and training choices.
5. Claude (Anthropic LLM). Used to help with structuring this report, debugging code, and ideating on TTA and ensemble strategies. The actual implementation, experiments, and analysis were done by us.

Team Contributions

Both team members contributed equally to this project. The workload was shared across model design, training, experimentation, and report writing.

Member	Contributions
Khwaja Abdul Samad (CS25MTECH11014)	Architecture design (SE-ResNet, SE block, skip connection logic), transform pipeline design, training loop implementation with milestone checkpointing, TTA inference pipeline, ensemble submission, and report writing (architecture, training, and ablation sections).
Sajid Ali (CS25MTECH11019)	Architecture design (joint), training loop debugging, checkpoint evaluation (Cell 11 evaluation code), ensemble logic, hyperparameter decisions (batch size, LR, dropout), and report writing (introduction, results, and conclusion sections).

Table 8: Contributions from each team member.